MANTLE TURING TEST HACKATHON 2026

AI x RWA TRACK

Comprehensive Team Build Guide

Dynamic Yield Strategies & Automated Risk Management
for USDY, mETH & Mantle RWA Infrastructure

Build Period	Prize Pool
28 Days	$100,000

Phase 2: AI Awakening | Team Edition | May 2026

1. AI & Real-World Assets (RWA): Overview & Market Significance

1.1 What Are Real-World Assets (RWA)?

Real-World Assets are tokenized representations of off-chain, tangible or financial assets placed on a blockchain. They bridge the multi-trillion-dollar traditional financial world with decentralised protocols, enabling previously illiquid assets to be traded, collateralised, and yield-generating 24/7.

Examples of RWAs span a wide spectrum:

- Yield-bearing stablecoins backed by US Treasuries (e.g., Ondo Finance's USDY)
- Liquid staking tokens representing staked ETH positions (e.g., Mantle's mETH)

- Tokenised real estate, corporate bonds, and trade finance receivables
- Commodity-backed tokens (gold, oil, carbon credits)

1.2 Market Statistics & Significance

The RWA sector is experiencing explosive growth, making it one of the most strategically important areas in DeFi:

Metric	Data Point
Total RWA Market Cap (2025)	~$20 billion+ on-chain (up from ~$5B in 2023)
US Treasury Token Market	>$3 billion in tokenized T-Bills across major protocols
Institutional Participation	BlackRock, Franklin Templeton, JPMorgan all active in RWA tokenization
Projected RWA Market (2030)	$10–16 trillion according to Boston Consulting Group
Mantle Network TVL	>$1 billion in TVL anchored by RWA-backed assets
USDY (Ondo Finance)	Yield-bearing stablecoin backed by short-term US Treasuries + bank deposits
mETH (Mantle)	Liquid staking token for ETH deposited into Mantle's staking protocol

1.3 What Is AI x RWA?

The AI x RWA convergence refers to the application of artificial intelligence — including machine learning, reinforcement learning, and agentic frameworks — to autonomously manage, optimise, and protect RWA portfolios on-chain. This track specifically challenges builders to create AI agents that:

- Monitor yield rates and rebalance between USDY, mETH, and other RWA instruments autonomously
- Detect anomalies, market stress signals, and smart-money flows in real time
- Execute risk management actions (position reduction, hedging, liquidation prevention) with no human in the loop
- Log all decisions permanently on Mantle's blockchain for full auditability

Why This Track Matters

AI x RWA is not just a technical exercise. It directly addresses a trillion-dollar market inefficiency: most RWA yield strategies today are managed manually or by static smart contracts. An AI that can dynamically adapt to interest rate changes, liquidity shifts, and on-chain risk signals is a genuinely novel and commercially valuable product — exactly what the Mantle Turing Test judges are looking for.

2. Dynamic Yield Strategies & Automated Risk Management

2.1 What Is a Dynamic Yield Strategy?

A static yield strategy simply deposits assets into a single protocol and earns its advertised APY. A dynamic yield strategy continuously monitors multiple yield opportunities across protocols and rebalances allocations based on changing rates, risk, and liquidity conditions — all without human intervention.

Key Concepts

Concept	Definition
Yield Optimisation	Maximising APY by automatically moving capital between USDY pools, mETH staking, lending protocols (e.g., Aave on Mantle), and DEX LPs
Rebalancing Triggers	Predefined conditions (APY threshold, risk score, liquidity ratio) that instruct the AI agent to reallocate funds
Compounding	Automatically harvesting and reinvesting yield rewards to maximise long-term returns
Risk-Adjusted Yield	Weighing expected APY against protocol risk, smart contract risk, and market volatility before allocation
Capital Efficiency	Using flash loans or collateral recycling to maximise yield without increasing actual capital deployed

Example: Dynamic USDY/mETH Strategy

Imagine your AI agent has $100,000 to manage. At T=0, USDY pools on Merchant Moe offer 7.2% APY while mETH staking yields 4.8%. The agent allocates 70% to USDY and 30% to mETH. Three days later, US Treasury rates drop and USDY yield falls to 5.1%, while Agni Finance launches a new mETH/ETH LP incentive at 11%. The agent detects this shift, calculates the risk-adjusted return differential, and moves 40% of the USDY allocation into the new LP — all on-chain, with the decision recorded via ERC-8004.

2.2 What Is Automated Risk Management?

Automated risk management is a system that continuously evaluates the health and safety of on-chain positions and executes protective actions automatically when predefined risk thresholds are breached. For RWA assets, this is critical because:

- RWA-backed tokens can de-peg from their underlying asset under stress conditions
- DeFi protocols can be exploited, causing sudden liquidity crises
- Interest rate movements in TradFi directly impact RWA token yields and valuations

Risk Management Layers for Your Project

Risk Type	Description	Mitigation Strategy
Protocol Risk	Smart contract exploit in a yield protocol	Set maximum allocation per protocol (e.g., 25% cap); monitor TVL and audit status
Liquidity Risk	Inability to exit a position quickly enough	Track on-chain liquidity depth; set minimum liquidity thresholds before entry
De-peg Risk	USDY or mETH deviating from expected peg	Oracle monitoring with automated exit triggers at >0.5% sustained deviation
Concentration Risk	Over-exposure to a single asset or protocol	Enforce diversification rules across at least 3 protocols at all times
Oracle Manipulation	Price feeds being manipulated to trigger false signals	Use TWAP (time-weighted average price) instead of spot prices; cross-reference multiple oracles
Gas/Execution Risk	Transactions failing during volatile periods	Implement gas price caps and fallback execution paths

3. Setting Up Mantle RWA Infrastructure

3.1 Understanding Mantle's Architecture

Mantle is an EVM-compatible Layer 2 blockchain built on Ethereum using Optimistic Rollup technology. Its key distinguishing features for your project are:

- Native RWA integration: USDY and mETH are first-class citizens in Mantle's DeFi ecosystem
- Low gas costs (~10-50x cheaper than Ethereum L1), enabling frequent AI-driven rebalancing transactions
- ERC-8004 standard: Your AI agent will receive a unique on-chain identity NFT — every decision it makes is permanently recorded
- Integrated DeFi protocols: Merchant Moe (DEX), Agni Finance (concentrated liquidity), and Fluxion (order book DEX)

3.2 Development Environment Setup (Step-by-Step)

Step 1: Install Prerequisites

Install the following tools before writing any code:

Tool	Install Command / Action

Node.js v20+	npm install -g node (or use nvm)
Python 3.11+	Required for AI/ML components and data pipelines
Foundry (Solidity toolkit)	curl -L https://foundry.paradigm.xyz | bash && foundryup
Hardhat (alternative)	npm install --save-dev hardhat
MetaMask wallet	Install browser extension; create a dedicated dev wallet
Git + GitHub	For version control and team collaboration

Step 2: Configure Mantle Network

Add Mantle Mainnet and Testnet to your development environment:

Parameter	Value
Network Name	Mantle Mainnet / Mantle Sepolia Testnet
Chain ID (Mainnet)	5000
Chain ID (Testnet)	5003
RPC URL (Mainnet)	https://rpc.mantle.xyz
RPC URL (Testnet)	https://rpc.sepolia.mantle.xyz
Block Explorer	https://explorer.mantle.xyz
Native Token	MNT (for gas fees)
Faucet (Testnet)	https://faucet.testnet.mantle.xyz

Step 3: Set Up Your Project Repository

Create a monorepo structure that separates your AI agent logic from your smart contract layer:

Recommended Project Structure

rwa-ai-agent/ contracts/ <- Solidity smart contracts (strategy vault, risk manager) agent/ <- Python AI agent (decision engine, model training) models/ <- ML models for yield prediction & risk scoring strategies/ <- Strategy implementations (yield optimiser, risk module) data/ <- Data pipelines (oracle feeds, on-chain analytics) frontend/ <- Dashboard UI (React/Next.js) tests/ <- Unit + integration tests scripts/ <- Deployment and monitoring scripts .env <- API keys and private keys (NEVER commit this file)

Step 4: Connect to Key Protocol Contracts

Your AI agent will need to interact with these smart contracts on Mantle. Obtain their official addresses from Mantle's documentation and verified block explorer:

Contract	Where to Find Verified Address

USDY Token Contract	Ondo Finance's USDY deployed on Mantle — check docs.ondo.finance for the verified address
mETH Token Contract	Mantle's liquid staking token — check docs.mantle.xyz/meth for the verified address
Merchant Moe Router	DEX router for swaps and LP management — see merchantmoe.com/docs
Agni Finance Pool Factory	Concentrated liquidity pools — see agni.finance/docs
Fluxion Contract	Order book DEX on Mantle — see fluxion.fi/docs
OraKle Oracle	Real-time price feeds endorsed by Mantle hackathon partners
Pyth Network Oracle	Cross-chain price feeds (backup oracle source)

Step 5: Obtain API Keys

- Bybit API: Register at bybit.com for trading data and execution support (provided to hackathon participants)
- Nansen API: On-chain analytics and smart money tracking — mantle hackathon participants get access
- OraKle API: Real-time oracle data partner for the hackathon
- Tencent Cloud: Cloud compute for AI model training and inference (hackathon sponsor)
- Alchemy or Infura: Backup RPC provider for reliable Mantle node access

3.3 Smart Contract Architecture

Your project should consist of at least three core smart contracts:

Contract	Purpose
StrategyVault.sol	Holds user funds, executes deposit/withdraw, calls strategy modules. This is the entry point for all capital.
YieldOptimiser.sol	Contains the logic for allocating capital across USDY and mETH pools. Receives instructions from the AI agent via signed transactions.
RiskManager.sol	Monitors position health, checks oracle prices, and can trigger emergency withdrawals or rebalancing when risk thresholds are breached.
AgentIdentity.sol (ERC-8004)	Issues your AI agent its on-chain identity NFT. All agent actions are attributed to this identity — critical for hackathon scoring.

4. Security Best Practices & Risk Management

4.1 Smart Contract Security

Smart contract vulnerabilities can result in total loss of funds. For a hackathon project handling real RWA assets, security must be built in from day one — not added at the end. Follow these practices rigorously:

Critical Vulnerability Classes to Defend Against

Vulnerability	Mitigation Strategy
Reentrancy Attack	Follow the Checks-Effects-Interactions (CEI) pattern. Use OpenZeppelin's ReentrancyGuard modifier on all external-facing functions.
Oracle Manipulation	Never rely on a single oracle. Use Chainlink or OraKle as primary; cross-validate with Pyth. Use TWAP (30-minute minimum) instead of spot prices.
Flash Loan Attacks	Implement per-block transaction limits. Use the 'commit-reveal' scheme for sensitive price-dependent operations.
Access Control Flaws	Use OpenZeppelin's AccessControl or Ownable. Every admin function must require explicit role. Never use tx.origin for auth.
Integer Overflow/Underflow	Use Solidity 0.8+ which has built-in overflow protection, or SafeMath for older versions.
Front-Running	For yield strategy rebalancing, use time-locks (minimum 1 block delay) or Mantle's private mempool features if available.
Unbounded Loops	Cap all loops with a maximum iteration count to prevent out-of-gas denial of service attacks.

4.2 AI Agent Security

Your AI agent is a novel attack surface that traditional smart contract security doesn't cover. Consider these threats:

- Adversarial inputs: Manipulated oracle data can trick your model into making bad rebalancing decisions — implement sanity checks on all incoming data
- Model poisoning: If you use off-chain training data, validate the data source and implement outlier detection
- Execution safety: Your agent should have a maximum transaction size limit per hour to prevent runaway behaviour
- Circuit breakers: Implement an on-chain emergency stop that any team member (multi-sig) can trigger to pause all agent actions
- Slippage protection: All DEX transactions must have a maximum slippage parameter (recommend 0.5% for stable pairs, 2% for volatile pairs)

4.3 Operational Security

- Use a hardware wallet (Ledger/Trezor) for any mainnet operations

- Store all private keys in environment variables — never hardcode in source code
- Use a .gitignore that explicitly excludes .env files before making your repo
- Implement multi-signature (multi-sig) for any admin functions using Gnosis Safe
- Deploy to Mantle Testnet and run for at least 72 hours before any mainnet deployment
- Keep all dependencies updated — audit package.json and requirements.txt for known CVEs

Security Checklist

Before final submission, verify: [ ] All contracts compiled with Solidity 0.8.20+ [ ] OpenZeppelin ReentrancyGuard on all state-changing functions [ ] Two independent oracle sources configured [ ] Emergency pause (circuit breaker) implemented and tested [ ] No hardcoded private keys or API keys in codebase [ ] All admin functions protected by multi-sig or at minimum 2-of-4 team keys [ ] Slippage protection on all DEX interactions [ ] Unit test coverage > 80% [ ] At least 48 hours of testnet operation logged [ ] Agent decision logs verified on-chain via ERC-8004

5. Recommended Tools & Technologies

5.1 Smart Contract Development

Tool	Purpose & Notes
Solidity 0.8.20+	Primary language for all smart contracts on Mantle (EVM-compatible)
Foundry (forge, cast, anvil)	Best-in-class Solidity testing and deployment framework. Use forge test for unit tests, anvil for local node.
Hardhat	Alternative to Foundry; excellent for JavaScript-based testing and deployment scripts
OpenZeppelin Contracts	Battle-tested security libraries: AccessControl, ReentrancyGuard, SafeERC20, Pausable
Slither	Python-based static analyser for Solidity — run before every deployment
Mythril	Symbolic execution security scanner for smart contracts
Tenderly	Real-time smart contract monitoring, debugging, and alerting on Mantle

5.2 AI / Machine Learning

Tool	Purpose & Notes
Python 3.11+	Primary language for all AI/agent components
PyTorch or TensorFlow	Deep learning frameworks for yield prediction models

Scikit-learn	Classical ML for risk scoring (Random Forest, Gradient Boosting, Isolation Forest for anomaly detection)
Stable Baselines 3	Reinforcement learning library — use PPO or SAC for dynamic rebalancing agent training
Gymnasium (OpenAI Gym)	Create a custom DeFi environment for training your RL agent off-chain before deployment
Pandas + NumPy	Data manipulation for on-chain time series processing
LangChain / CrewAI	Agent orchestration frameworks for building multi-step decision pipelines
FastAPI	Serve your AI model inference as a microservice that the smart contract oracle can query

5.3 Data & Analytics

Tool	Purpose & Notes
Nansen	Smart money wallet tracking; identify whale movements in USDY/mETH pools before they happen
OraKle	Official hackathon oracle partner; real-time on-chain price and yield data for Mantle
Dune Analytics	Custom SQL queries on Mantle chain data; excellent for backtesting your strategy on historical data
The Graph	Index and query on-chain events from Merchant Moe, Agni Finance using GraphQL
Pyth Network	Cross-chain oracle providing real-time pricing for 400+ assets including ETH, USDC, and RWA tokens
Mantle Explorer API	Direct access to Mantle block data, transaction history, and contract events

5.4 Frontend & Infrastructure

Tool	Purpose & Notes
Next.js 14 + React	Framework for your monitoring dashboard — shows live yield, agent decisions, and risk metrics
wagmi + viem	Type-safe Ethereum/Mantle React hooks for wallet connection and contract interaction
TailwindCSS	Utility-first CSS framework for rapid dashboard UI development
Recharts / Chart.js	Real-time yield and performance charts in your dashboard
Tencent Cloud	Hackathon sponsor cloud provider — use for AI model training compute and API hosting
Docker	Containerise your agent and API services for reproducible deployment

GitHub Actions	CI/CD pipeline: auto-run tests on every push, auto-deploy to testnet on merge to main

6. Case Studies & Successful Implementations

6.1 Yearn Finance — The Original Yield Optimiser

Yearn Finance pioneered automated yield optimisation in DeFi, and its architecture heavily influences the AI x RWA space. Yearn's 'Vaults' automatically move capital between Compound, Aave, and Curve Finance based on yield differentials.

- Key lesson: Modular strategy architecture allows adding new yield sources without rewriting core vault logic — follow this pattern in YieldOptimiser.sol
- Key lesson: Yearn's early v1 suffered a $11M exploit due to a flash loan attack on price oracles — exactly why TWAP oracle validation is non-negotiable
- Outcome: Yearn achieved >$7B TVL at peak by making yield optimisation accessible to non-technical users

6.2 Ondo Finance — USDY & Institutional RWA

Ondo Finance tokenized US Treasury yields into USDY, making institutional-grade fixed income accessible on-chain. USDY is one of the two primary assets in your track.

- Key lesson: RWA tokens require real-time oracle feeds that reflect underlying asset NAV (net asset value) — your risk manager must track USDY's peg against its reference basket
- Key lesson: Ondo maintains a redemption buffer so large withdrawals don't destabilise the peg — your vault should simulate this with a withdrawal queue
- Outcome: USDY grew to >$500M in circulating supply within 18 months of launch

6.3 Gauntlet Network — On-Chain Risk Management AI

Gauntlet is arguably the most successful AI risk management system in DeFi. It builds simulation models that generate parameter recommendations (borrow caps, collateral factors, liquidation thresholds) for Aave, Compound, and other protocols.

- Key lesson: Gauntlet uses agent-based simulation to stress-test portfolios under extreme market scenarios (e.g., ETH -80% in 24 hours) before deploying risk parameters
- Key lesson: Their 'Economic Safety' reports are transparent and auditable — the Mantle hackathon's on-chain ERC-8004 decision logging is inspired by this philosophy
- Outcome: Gauntlet has managed >$15B in DeFi risk parameters and prevented an estimated $1B+ in bad debt

6.4 Sommelier Finance — Autonomous Strategy Vaults

Sommelier builds 'Real Yield' vaults that use off-chain AI models to manage on-chain DeFi positions, bridging the architecture gap your project needs to build.

- Key lesson: Sommelier uses a 'Strategist' role — an off-chain AI that submits rebalancing transactions to the vault contract — exactly the pattern to follow for your agent-contract interface
- Key lesson: All strategist actions are validated by on-chain checks (slippage limits, position size caps) before execution — the on-chain safety layer is essential
- Outcome: Their USDC vault delivered consistent 8–15% APY through the 2022-2023 bear market by dynamically managing Uniswap v3 concentrated liquidity

7. 28-Day Build Plan: Milestones & Timeline

Phase Overview

Phase	Focus
Phase 1 (Days 1–5)	Foundation & Research — Learn the ecosystem, set up dev environment, define project scope
Phase 2 (Days 6–14)	Core Build — Smart contracts + AI agent skeleton + data pipeline
Phase 3 (Days 15–21)	Integration & Intelligence — Connect AI to contracts, train models, deploy to testnet
Phase 4 (Days 22–28)	Polish & Testing — Security audit, UI dashboard, documentation, submission prep

Detailed Milestone Table

Days	Focus Area	Owner	Key Tasks	Deliverable
1-2	Environment & Research	All 4	Install tools, create repo, configure Mantle testnet, read USDY/mETH docs	Dev environment ready; test wallet funded
3-4	Protocol Deep Dive	All 4	Interact with Merchant Moe, Agni Finance, Fluxion via scripts; understand APY calculation	Working script that reads live yields from all 3 protocols

5	Architecture Design	All 4	Define contract interfaces, AI agent architecture, data flow diagram	Architecture doc + flow diagram signed off by team
6-8	Smart Contracts v1	Dev 1+2	Write StrategyVault.sol, YieldOptimiser.sol, RiskManager.sol; unit tests	Contracts deployed to local Anvil node, 60% test coverage
9-11	AI Agent Skeleton	Dev 3+4	Python agent structure, Mantle RPC connection, read yield data, basic decision logic	Agent reads live data, logs decisions, no execution yet
12-14	Data Pipeline	Dev 3+4	OraKle + Nansen integration, historical data collection, feature engineering for ML	Dataset of 90-day yield + risk metrics ready for training
15-17	ML Model Training	Dev 3+4	Train yield prediction model (LSTM or XGBoost), risk scoring model (Isolation Forest)	Models with >70% directional accuracy on test set
18-19	Agent-Contract Integration	All 4	Agent signs + submits rebalancing txs to testnet contracts; ERC-8004 identity setup	First successful AI-driven rebalance on Mantle Sepolia testnet
20-21	Risk Manager Activation	Dev 1+2	Implement circuit breakers, oracle validation, emergency pause; full risk module testing	Risk module triggers correctly on simulated de-peg scenario
22-23	Security Audit Pass	All 4	Run Slither + Mythril, fix all HIGH/MEDIUM findings, peer code review	Zero HIGH severity findings; security checklist completed
24-25	Frontend Dashboard	Dev 3+4	Build Next.js dashboard: live yield, agent actions, risk metrics, position breakdown	Live dashboard connected to testnet showing real agent data
26	48-Hour Testnet Run	All 4	Agent runs continuously on testnet; monitor for bugs, edge cases, gas issues	48-hour log with all transactions verified on-chain
27	Documentation	All 4	Write README, technical docs, architecture diagrams, demo video script	Complete docs + 5-min demo video recorded
28	Final Submission	All 4	Final review, deploy to submission environment, submit on DoraHacks with all materials	Submitted on DoraHacks before deadline

8. Actionable Starting Steps for Day 1

8.1 First Research Topics (Priority Order)

1. Read the Mantle documentation: docs.mantle.xyz — focus on the RWA section, mETH staking, and developer quickstart
2. Understand USDY: Read Ondo Finance's USDY whitepaper at ondo.finance — learn how its yield accrual mechanism works
3. Study ERC-8004: Understand Mantle's agent identity standard — every action your agent takes will be attributed to its ERC-8004 NFT
4. Explore Merchant Moe and Agni Finance: Use their testnets to manually execute swaps and provide liquidity — understanding the UX will inform your AI's decision logic
5. Read the Sommelier Finance architecture blog post: This is the closest public reference to your target system
6. Study Gauntlet's risk methodology paper: Even a high-level read will give you the mental models for on-chain risk scoring

8.2 Learning Resources

Blockchain & Mantle

Resource	Details
Mantle Developer Docs	docs.mantle.xyz — Official documentation for all Mantle contracts and APIs
Mantle YouTube Channel	@0xMantle on YouTube — Technical tutorials and ecosystem overviews
Ondo Finance Docs	docs.ondo.finance — USDY mechanics, redemption, and oracle integration
Merchant Moe Docs	docs.merchantmoe.com — DEX integration guide for yield strategies

Smart Contract Development

Resource	Details
Foundry Book	book.getfoundry.sh — Complete guide to Foundry, the recommended dev toolkit
OpenZeppelin Docs	docs.openzeppelin.com — Security contracts reference (use AccessControl, ReentrancyGuard)
Solidity by Example	solidity-by-example.org — Practical patterns for DeFi contracts
Smart Contract Security by Trail of Bits	github.com/crytic/building-secure-contracts — Free in-depth security guide

AI & Machine Learning for DeFi

Resource	Details
Stable Baselines 3 Docs	stable-baselines3.readthedocs.io — RL library documentation with DeFi-applicable examples
Hugging Face Time Series Course	huggingface.co/learn — Time series forecasting for yield prediction
LangChain Documentation	python.langchain.com — Build AI agent pipelines with tool-use capabilities
DeFi x AI Research Papers	arxiv.org — Search 'DeFi reinforcement learning' for academic references to cite

8.3 Team Collaboration & Task Delegation

Recommended Role Split (4-Person Team)

Role	Responsibilities
Team Member 1: Smart Contract Lead	Owns StrategyVault.sol, YieldOptimiser.sol, RiskManager.sol. Responsible for security audit and deployment. Primary tool: Foundry.
Team Member 2: Blockchain Integration	Owns contract deployment scripts, testnet management, Mantle RPC integration, and ERC-8004 agent identity setup.
Team Member 3: AI/ML Engineer	Owns Python AI agent, ML model training (yield prediction + risk scoring), and agent decision logic.
Team Member 4: Data & Frontend	Owns OraKle/Nansen data pipelines, on-chain data processing, and the Next.js monitoring dashboard.

Daily Team Sync Structure

- Morning standup (15 min): What did you complete yesterday? What is your focus today? Any blockers?
- Shared Notion or Linear board: Keep a running task list with owners and due dates visible to all
- GitHub PR reviews: No code merges to main without at least one teammate review — this catches bugs early
- End-of-day push: Everyone commits and pushes their work before end of day to prevent lost work
- Weekly demos (Days 7, 14, 21, 28): Each team member demos their component — this builds shared understanding and catches integration issues

Communication & Tools

- Discord or Telegram group for async communication — create channels: #general, #smart-contracts, #ai-agent, #bugs, #submissions
- GitHub Projects for task tracking — use issues for bugs and PRs for all code changes
- Shared Google Drive or Notion for research notes, architecture docs, and meeting notes

- Loom for async video updates when something is too complex to explain in text

9. Judging Criteria & Strategy to Win

9.1 What Judges Will Look For

Based on the hackathon's emphasis on the 'Human vs. AI mechanism' and on-chain benchmarking, judges will evaluate:

Criterion	What This Means for Your Build
On-Chain Verifiability	Every AI decision must be traceable on Mantle via ERC-8004. This is non-negotiable — make sure every rebalancing action is logged.
Risk-Adjusted Returns	Not just raw yield — judges want to see your agent maximise Sharpe ratio (return per unit of risk), not just chase the highest APY recklessly.
Technical Sophistication	Use of ML/RL beyond simple if-then rules. A neural network or trained RL agent beats a hardcoded threshold strategy.
Security & Robustness	Can your system handle edge cases? Simulate a de-peg event during your demo. Show your circuit breakers working.
Innovation	Novel approach to USDY/mETH yield management. What does your AI do that a static smart contract cannot?
Presentation & Demo	The live stream component means your demo must be visual, clear, and compelling. Build a dashboard.

9.2 Differentiating Your Submission

To stand out from other teams, consider these differentiators:

- Implement a 'Backtesting Mode': Show historical performance of your strategy on 90+ days of on-chain data before it touches real funds
- Add 'Explainable AI' to your dashboard: Show WHY the agent made each decision (feature importance, confidence score) — this is rare and impressive
- Build a cross-protocol risk score: An aggregate health score that combines USDY peg stability, mETH staking queue depth, and protocol TVL trends
- Simulate adversarial scenarios in your demo: Show your risk manager successfully halting the agent during a simulated USDY de-peg — judges love live safety demonstrations
- Leverage the ERC-8004 leaderboard: Your agent's on-chain track record IS your proof of performance — optimise for this metric from Day 6 onward

10. Quick Reference Card

Key Addresses & Links

Resource	URL
Mantle RPC (Testnet)	https://rpc.sepolia.mantle.xyz
Mantle Explorer	https://explorer.mantle.xyz
Testnet Faucet	https://faucet.testnet.mantle.xyz
Hackathon Page	https://dorahacks.io/hackathon/mantleturingtesthackathon2026/detail
Mantle Developer Docs	https://docs.mantle.xyz
Ondo USDY Docs	https://docs.ondo.finance
Merchant Moe Docs	https://docs.merchantmoe.com
Agni Finance Docs	https://docs.agni.finance
OpenZeppelin Contracts	https://docs.openzeppelin.com/contracts
Foundry Book	https://book.getfoundry.sh

Emergency Contacts & Support

- Mantle Discord: discord.gg/0xMantle — Use the #developer-support channel for technical questions
- DoraHacks Hackathon Discord: Access via your hackathon registration confirmation email
- Byreal Support: For RealClaw platform and Bybit API integration issues

Good luck, team. Build something that makes judges say: no human could have done this alone.

Mantle Turing Test Hackathon 2026 | AI x RWA Track | $100,000 Prize Pool